Strings

08 July 2026 20:26

String is a sequence which is made up of one or more Unicode characters. Here the character can be a letter, digit, white space or any other symbol.
A string can be created by enclosing one or more characters in a single, double, or triple quote.

```
print('\u20b9')
₹
print('\u0048')
H
print('\u0040')
@
print('\u0041')
A
```

Unicode values, they are 4 digit hexadecimal values.

```
str1 = 'Hello! python'
str2 = "Shiv Ray"
Str3 = """Hello shiv
How r u?
Are you coding!
"""
```

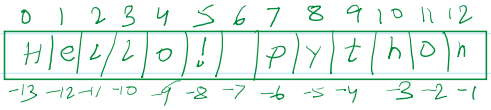
← Triple quote is used for multi line string.

```
Str3
'Hello shiv\nHow r u?\nAre you coding!\n'
print(Str3)
Hello shiv
How r u?
Are you coding!
```

character inserted by Python when we use triple quote.

```
str1
'Hello! python'
str2
'Shiv Ray'
Str3
```

Internally Python holds the string into single quotes.




```
str4 = '''I am coding
in Python
not in Java'''
str4
'I am coding\nin Python\nnot in Java'
print(str4)
I am coding
in Python
not in Java
```

Accessing characters in a string

Each individual character in a string can be accessed using a technique called indexing. The index specifies the character to be accessed in the string and is written in square brackets([]). The index of the first character (from left) In the string is zero and the

last character is N minus 1. Where N is the length of the string. If we give index value out of this range then we get an index error. The index must be an integer (positive, 0 or negative).
'0' index represents the first character of the string and '-1' index represents the last character of the string.
For example:

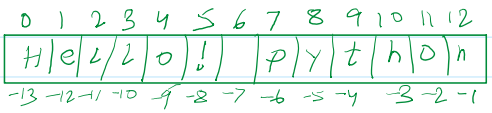
```
str1
'Hello! python'
str1[0]
'H'
str1[1]
'e'
str1[2]
'l'
str1[-1]
'n'

len(str1)
13
str1[20]
Traceback (most recent call last):
  File "<pyshell#26>", line 1, in <module>
    str1[20]
IndexError: string index out of range
str1[-20]
Traceback (most recent call last):
  File "<pyshell#27>", line 1, in <module>
    str1[-20]
IndexError: string index out of range
```

first character

last character

total number of characters in the string

Python allows an index value to be negative also. Negative indices are used when we want to access the character of the string from right to left. Starting from right hand side, the first character has the index as -1 and the last character has the index -N. Where N is the length of the string.



Accessing the first character and the last character in another way:

```
n = len(str1)
n
13
str1[n-1]
'n'
str1[-n]
'H'
str1
'Hello! python'
```

String is Immutable
A string is an immutable data type. It means that the

content of the string cannot be changed after it has been created. An attempt to do this would lead to an error.

```
str1
'Hello! python'
str1[0] = 'h'
Traceback (most recent call last):
  File "<pyshell#35>", line 1, in <module>
    str1[0] = 'h'
TypeError: 'str' object does not support item
assignment
```

String Operations

As we know that String is a sequence of characters. Python allows certain operations on string data types, such as concatenation, repetition, membership, and slicing.

Concatenation: To concatenate means to join. Python allows us to join two strings using concatenation operator which is denoted by symbol +.

Example:

```
fname = 'Shiv'
lname = 'Ray'
fullName = fname + ' ' + lname
fname
'Shiv'
lname
'Ray'
fullName
'Shiv Ray'
```

Repetition

Python allows us to repeat the given string using repetition operator which is denoted by symbol *.

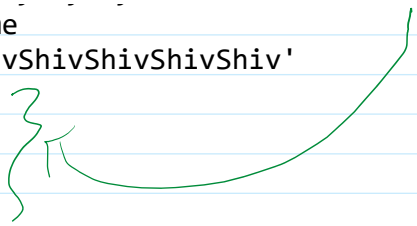
For example:

```
fname
'Shiv'
lname
'Ray'
fullName
'Shiv Ray'
char = '%'
newchars = char * 10
newchars
'%%%%%%%%%%'
lname * 5
'RayRayRayRayRay'
6 * fname
'ShivShivShivShivShivShiv'
char
```

For repetition:
one operand must be of type 'str' and other operand must be of type 'int'.

It will not change the original variable. The data will remain as it is.

```
6 * fname
'ShivShivShivShivShivShiv'
char
'%'
fname
'Shiv'
```



Membership: Python has two membership operators, **'in'** and **'not in'**. The 'in' operator takes two strings and returns True if the first string appears as a substring in the 2nd string, otherwise it returns False.
For example:

```
str1
'Hello! python'
'o' in str1
True
'y' in str1
True
'P' in str1
False
'Py' in str1
False
'py' in str1
True
```

The **'not in'** operator also takes two strings and returns True if the 1st string does not appear as the substring in the 2nd string. Otherwise returns False.
For example:

```
str1
'Hello! python'
'x' not in str1
True
'p' not in str1
False
'py' not in str1
False
'pyc' not in str1
True
```